PayFlow

Payment Gateway System

Technical Project Report

Language TypeScript	**Runtime** Node.js
Pattern Singleton + Factory	**Principles** All 5 SOLID

Database: PostgreSQL (NeonDB) | Deployment: Vercel | Frontend: Vanilla JS

Project Team

Name	Enrollment No.
Adarsh	2401010026
Ankit	2401010073
Anshika	2401010080
Himanshu	2401010190
Lakshya	2401010253

1. Executive Summary

PayFlow is a scalable, multi-vendor Payment Gateway System built entirely with TypeScript and Node.js. It demonstrates advanced software engineering principles including Object-Oriented Programming (OOP), the SOLID design principles, and classic Gang-of-Four design patterns — all within a practical, real-world payment context.

The system is architecturally layered into three distinct tiers: a TypeScript/Express REST API backend, a Vanilla JavaScript single-page frontend, and a serverless PostgreSQL database hosted on NeonDB. Both the frontend and backend are deployed on Vercel, making the system fully cloud-native and production-accessible.

> **Project Significance**
> PayFlow serves as a comprehensive reference implementation for beginners and intermediate developers who want to see OOP, SOLID principles, and design patterns applied in a real, running application — not just in textbook examples.

Live Deployment

Layer	URL
Frontend	payflow-frontend (Vercel)
Backend API	payflow-backend (Vercel)
Database	NeonDB — PostgreSQL Serverless

Key Highlights

- Full-stack TypeScript application with REST API and SPA frontend
- Implements Singleton and Factory design patterns
- Demonstrates all four OOP pillars: Encapsulation, Abstraction, Inheritance, and Polymorphism
- Adheres to all five SOLID principles across the codebase
- Automated test suite with 75+ test scenarios
- Three demo user accounts pre-loaded for testing

2. Technology Stack

PayFlow deliberately uses a focused, well-understood technology stack that maximises learning clarity without sacrificing production realism.

Layer	Technology	Purpose
Backend	TypeScript 5.4, Node.js, Express	Type-safe REST API server
Frontend	HTML5, CSS3, Vanilla JavaScript	Single-page application shell
Database	PostgreSQL via NeonDB Serverless	Persistent transaction & user storage
Deployment	Vercel (frontend + backend)	Cloud-native CI/CD deployment
Runtime Tools	ts-node, tsc	Direct TS execution & compilation
Testing	Custom TestCases.ts runner	75+ automated test scenarios
Fonts	Google Fonts (Sora, DM Sans, JetBrains Mono)	Professional UI typography

TypeScript Configuration Highlights

The tsconfig.json is configured with strict: true, enabling the strongest compile-time guarantees. This means incorrect types — such as passing a string where a payment amount (number) is expected — are caught before runtime, long before any user is impacted.

Setting	Value	Effect
target	ES2020	Modern JavaScript output with all current features
strict	true	Maximum type safety — runtime errors caught at compile time
module	commonjs	Node.js compatible module system
declaration	true	Generates .d.ts files for reuse in other projects
sourceMap	true	Enables line-accurate debugging of TypeScript source

3. System Architecture & Design

PayFlow follows a clean layered architecture with clearly defined responsibilities at every level. Each layer communicates through well-defined contracts (interfaces), ensuring loose coupling throughout the system.

3.1 Directory Structure

Folder / File	Responsibility
src/models/	All data classes — User, BaseTransaction, Transaction, UPIPayment, NetBankingPayment
src/interfaces/	TypeScript contracts defining what each module must implement
src/gateway/	PaymentGateway Singleton — the central transaction coordination engine
src/factory/	TransactionFactory — creates the right payment object based on type
src/bank/	BankAPI — simulates bank debit, credit, and account validation operations
src/utils/	TransactionLogger — cross-cutting logging utility
src/Main.ts	Application entry point — Console UI and Facade over all subsystems
diagrams/	UML diagrams: Class, Use Case, Sequence, and ER

3.2 Class Hierarchy

The inheritance chain models the real-world concept that a UPI payment and a Net Banking payment are both specialised kinds of transactions:

Inheritance Chain

BaseTransaction (abstract) → Transaction → UPIPayment
→ NetBankingPayment

Class	Type	Key Responsibility
BaseTransaction	Abstract	Defines txnId, amount, timestamp, status and the abstract processPayment() contract
Transaction	Concrete (intermediate)	Adds sender, receiver, senderBank, receiverBank — holds User composition
UPIPayment	Concrete (leaf)	Overrides processPayment() with UPI ID validation and UPI channel logic
NetBankingPayment	Concrete (leaf)	Overrides processPayment() with IFSC verification and NEFT/RTGS logic

3.3 Interface Architecture

Three TypeScript interfaces form the contract layer of the system, implementing the Interface Segregation Principle:

Interface	Methods	Purpose
IPaymentProcessor	processPayment(), cancelPayment()	Operations that modify payment state
ITransactionQuery	getTransactionStatus(), getTransactionHistory()	Read-only query operations
IPaymentGateway	extends both above + registerUser(), getUserById()	Full contract implemented by PaymentGateway

4. Database Design (ER Diagram)

PayFlow's persistence layer is a PostgreSQL database hosted on NeonDB, a serverless Postgres provider. The schema models two primary entities — Users and Transactions — with a many-to-one relationship on both the sender and receiver sides.

4.1 Entity Overview

Entity	Key Fields	Notes
User	userId (PK), name, accountNumber, password, mpin, phone, balance	All sensitive fields (password, mpin) are never exposed directly — only via validate methods
Transaction	txnId (PK), senderId (FK), receiverId (FK), amount, type, status, timestamp	Polymorphic — extended by UPIPayment (upiId) and NetBankingPayment (ifscCode, netBankingId)
UPIPayment	txnId (FK), upiId	Stores the UPI-specific identifier linked to the parent transaction
NetBankingPayment	txnId (FK), ifscCode, netBankingId	Stores the IFSC and net banking identifiers

4.2 Relationships

- One User can SEND many Transactions (1:N relationship on senderId)
- One User can RECEIVE many Transactions (1:N relationship on receiverId)
- Each Transaction record has exactly one sender and one receiver
- UPIPayment and NetBankingPayment are specialised extensions of Transaction (table-per-hierarchy pattern)

4.3 In-Memory Data Structures

Within the running application, PayFlow stores active data using Map<string, T> collections instead of arrays. This is a deliberate scalability decision — a Map lookup by key runs in O(1) time, whereas scanning an array for a transaction ID would run in O(n). At millions of transactions, this difference is critical.

Structure	Key	Time Complexity	Use Case
Map<string, Transaction>	txnId	O(1)	Instant transaction status lookup and cancellation
Map<string, User>	userId	O(1)	Instant user authentication during

Structure	Key	Time Complexity	Use Case
			login

5. System Use Cases & Actor Interactions

PayFlow exposes seven distinct user-facing features. Every feature is orchestrated by the PaymentGateway Singleton and executed through the appropriate subsystem.

#	Feature	Description	Key Classes Involved
1	Register User	Create a new user with account details and an initial deposit balance	PaymentGateway, User
2	Login	Validate userId and password to establish a session	PaymentGateway, User.validatePassword()
3	Check Balance	Query the current account balance via BankAPI	BankAPI, User.getBalance()
4	Send Money (UPI)	Enter receiver, amount, and MPIN — validate, debit, credit, log	TransactionFactory, UPIPayment, BankAPI, TransactionLogger
5	Send Money (Net Banking)	Same flow as UPI with additional IFSC code verification	TransactionFactory, NetBankingPayment, BankAPI
6	View Transaction History	Display all transactions retrieved from the PaymentGateway Map	PaymentGateway.getTransactionHistory ()
7	Cancel Transaction	Cancel a transaction only if its current status is PENDING	PaymentGateway.cancelPayment()

5.1 Demo User Accounts

Three pre-loaded accounts are available for immediate testing of all payment flows:

Name	User ID	Password	Account No.	MPIN	Starting Balance
Ankit	ankit01	pass123	ACC1001	1234	₹50,000.00
Priya	priya02	pass456	ACC1002	5678	₹30,000.00
Rahul	rahul03	pass789	ACC1003	9012	₹75,000.00

6. Object-Oriented Programming Concepts

PayFlow is built to be a textbook example of all four pillars of OOP applied in a production-grade codebase.

6.1 Encapsulation — User.ts

All fields in the User class are declared private. External code cannot read the MPIN or password directly — the only way to verify them is through the public validateMpin() and validatePassword() methods. This prevents accidental exposure of sensitive credentials anywhere in the codebase.

> **Design Intent**
> Encapsulation here is not just stylistic — it is a security boundary. If a future developer adds a feature that accidentally logs user objects, the sensitive fields will never appear in any log output because they are inaccessible by design.

6.2 Abstraction — BaseTransaction.ts

BaseTransaction is declared abstract, meaning no instance of it can ever be created directly. It defines the common data fields (txnId, amount, timestamp, status) shared by all payment types, and declares processPayment() as an abstract method — a contract that every subclass must fulfil with its own implementation.

The caller (PaymentGateway) never needs to know whether it is processing a UPI payment or a Net Banking payment. It simply invokes processPayment() and receives a boolean result. The complexity is fully hidden.

6.3 Inheritance — Transaction hierarchy

Class	Inherits From	What It Adds
Transaction	BaseTransaction	sender (User), receiver (User), senderBank, receiverBank, payment type
UPIPayment	Transaction	upiId field, UPI-specific processPayment() implementation
NetBankingPayment	Transaction	ifscCode, netBankingId, NEFT/RTGS processPayment() implementation

6.4 Polymorphism — Runtime dispatch

PaymentGateway stores all transactions as references to the abstract BaseTransaction type. When it calls transaction.processPayment(), the Java Virtual Machine (or in this case the V8 JavaScript engine) dispatches the call to the correct overridden implementation at runtime — UPIPayment or NetBankingPayment — without the gateway needing any if/else logic.

Instance at Runtime	processPayment() Behaviour
UPIPayment	Validates the UPI ID format → debit sender via BankAPI → credit receiver
NetBankingPayment	Verifies IFSC code → processes via NEFT/RTGS channel → debit/credit

7. SOLID Design Principles

All five SOLID principles are explicitly implemented across the PayFlow codebase. The table below provides a cross-reference, followed by a detailed explanation of each.

Principle	Where Applied	Benefit
S — Single Responsibility	User.ts, BankAPI.ts, TransactionLogger.ts each have one job	Isolated changes — modifying logging never risks breaking payments
O — Open/Closed	New payment type added by extending Transaction, not modifying it	Zero risk to existing UPI or Net Banking code when adding new types
L — Liskov Substitution	UPIPayment and NetBankingPayment are fully substitutable for Transaction	PaymentGateway works correctly with any future payment subclass
I — Interface Segregation	IPaymentProcessor and ITransactionQuery are separate interfaces	A reporting module only implements query methods — never forced to implement payment methods
D — Dependency Inversion	PaymentGateway depends on BaseTransaction abstraction, not UPIPayment	Decoupled architecture — adding CryptoPayment requires zero changes to PaymentGateway

7.1 Single Responsibility Principle (S)

Every class in PayFlow has exactly one reason to change. If the logging format needs to change, only TransactionLogger.ts changes. If bank validation logic changes, only BankAPI.ts changes. No class does more than one job.

Class	Its Single Responsibility
User.ts	Store and validate user credentials and balance
BankAPI.ts	Simulate bank-side debit, credit, and account validation
TransactionLogger.ts	Log transaction events (cross-cutting concern, isolated in one place)
TransactionFactory.ts	Decide which concrete payment class to instantiate
PaymentGateway.ts	Coordinate the overall payment flow between all subsystems

7.2 Open/Closed Principle (O)

Adding a new payment method — say, CryptoPayment — requires creating one new class that extends Transaction and implementing processPayment(). Adding a new case to TransactionFactory is the only change needed. BaseTransaction, Transaction, UPIPayment, NetBankingPayment, and PaymentGateway remain completely untouched.

7.3 Dependency Inversion Principle (D)

DIP in Action

PaymentGateway → depends on → BaseTransaction (abstraction) UPIPayment and NetBankingPayment → implement → BaseTransaction PaymentGateway never imports UPIPayment or NetBankingPayment directly. It never needs to.

8. Design Patterns

8.1 Singleton Pattern — PaymentGateway

The most architecturally critical constraint in PayFlow is that only one PaymentGateway instance must ever exist. If two gateway instances were running, each would have its own separate Map of transactions, and a transaction created through one gateway would be invisible to the other — causing catastrophic data inconsistency.

The Singleton pattern enforces this through three mechanisms: a private constructor (preventing external instantiation), a private static instance variable (holding the single reference), and a public static getInstance() method (which lazily creates the instance on first call, then returns the same object on every subsequent call).

Singleton Mechanism	Implementation
Private constructor	new PaymentGateway() fails outside the class — prevents accidental second instances
Static instance	private static instance: PaymentGateway | null = null
getInstance()	Creates instance only once; subsequent calls return the existing object
Real-world analogy	Like RBI (Reserve Bank of India) — one central bank coordinates all transactions

8.2 Factory Method Pattern — TransactionFactory

TransactionFactory centralises object creation. The caller (PaymentGateway or Main.ts) requests a transaction of a specific type and receives the correct concrete object — without importing or referencing UPIPayment or NetBankingPayment directly.

> **Factory Pattern — Key Benefit**
> When a new payment type (e.g. WalletPayment) is needed, only one new case is added to the factory switch statement. Every other class in the system — PaymentGateway, BankAPI, Logger — remains completely unchanged.

Benefit	Explanation
Decoupling	PaymentGateway never needs to know which concrete class it receives — only that it is a Transaction
OCP Support	New payment types extend the factory without modifying existing business logic

Benefit	Explanation
Single Point of Change	All payment type creation decisions live in one method — easy to audit and maintain
Testability	The factory can be mocked in unit tests to return controlled stub objects

8.3 Facade Pattern — Main.ts

Main.ts acts as a Facade over the entire system. From a user's perspective, they simply see a console menu with options. Behind that simple interface, Main.ts coordinates PaymentGateway (Singleton), TransactionFactory (Factory), BankAPI (Service), TransactionLogger (Utility), and User objects — all invisibly.

This is the same pattern used by modern payment SDKs like Razorpay or Stripe — a simple API surface that hides enormous internal complexity.

9. Core Workflow — Send Money Flow

The following table documents the step-by-step interaction between all system components when a user initiates a UPI payment. This maps directly to what the Sequence Diagram in the diagrams/ folder illustrates visually.

Step	Action	Class Responsible	Outcome
1	User enters userId and password	Main.ts	Input collected from console / frontend form
2	Password validated against stored hash	PaymentGateway + User.validatePassword()	User object returned if credentials match
3	User selects UPI as payment type and enters amount, receiver, MPIN	Main.ts	Payment parameters collected
4	MPIN validated	User.validateMpin()	Authorization confirmed before processing
5	TransactionFactory creates UPIPayment object	TransactionFactory.createTransaction('UPI', ...)	Correct concrete instance returned
6	PaymentGateway calls transaction.processPayment()	PaymentGateway → UPIPayment	Polymorphic dispatch to UPI-specific logic
7	UPI ID validated	UPIPayment.validateUpiId()	Transaction continues or aborts
8	Sender balance debited	BankAPI.debit(sender, amount)	Balance reduced; returns false if insufficient funds
9	Receiver balance credited	BankAPI.credit(receiver, amount)	Balance increased
10	Transaction logged	TransactionLogger.logTransaction()	Timestamped record written
11	Result displayed to user	Main.ts	Success or failure message shown

10. Class Diagram — Architecture

The full class diagram captures all classes, interfaces, attributes, methods, and relationships in the PayFlow system. The key relationships are summarised below.

10.1 Relationship Types

Relationship	Symbol	Example in PayFlow
Inheritance (IS-A)	△ (open triangle)	UPIPayment extends Transaction extends BaseTransaction
Implementation	△ (dashed)	PaymentGateway implements IPaymentGateway
Composition (owns)	◆ (filled diamond)	Transaction owns User objects — if transaction is deleted, users in that context are gone
Aggregation (references)	◇ (hollow diamond)	PaymentGateway references Users — users exist independently
Creation	→ (dashed arrow)	TransactionFactory creates UPIPayment or NetBankingPayment

10.2 Key Classes at a Glance

Class / Interface	Pattern	OOP Concept	SOLID
User.ts	—	Encapsulation	SRP
BaseTransaction.ts	Template Method	Abstraction	OCP
Transaction.ts	—	Inheritance, Composition	LSP
UPIPayment.ts	Strategy (implicit)	Polymorphism	LSP, OCP
NetBankingPayment.ts	Strategy (implicit)	Polymorphism	LSP, OCP
PaymentGateway.ts	Singleton	All 4 OOP Pillars	DIP, ISP
TransactionFactory.ts	Factory Method	Encapsulation	OCP, SRP
BankAPI.ts	—	Static utility methods	SRP
TransactionLogger.ts	Observer (simplified)	Static utility	SRP
Main.ts	Facade	All OOP indirectly	All SOLID indirectly
IPaymentProcessor.ts	Interface	Abstraction	ISP, DIP
ITransactionQuery.ts	Interface	Abstraction	ISP, DIP
IPaymentGateway.ts	Interface	Abstraction	ISP, DIP

11. Security & Performance

11.1 Security Measures

Security Concern	How PayFlow Addresses It
Password exposure	Password field is private; only validatePassword() method exposes a boolean result — never the raw value
MPIN exposure	Same encapsulation pattern — validateMpin() returns true/false only
MPIN authorization	Every payment requires MPIN validation before processPayment() is called — two-factor authorization flow
Insufficient funds	BankAPI.debit() checks balance before modifying it; returns false without throwing exceptions if funds are insufficient
Invalid transactions	Cancellation is only permitted on PENDING transactions — completed payments cannot be reversed through the cancel flow
Input validation	UPI payments validate the UPI ID format; Net Banking validates the IFSC code before any bank operation is attempted

11.2 Performance Considerations

- O(1) Map lookups ensure that transaction retrieval scales independently of the total number of transactions
- The Singleton pattern prevents redundant object creation and guarantees a single, consistent transaction store
- TypeScript strict mode eliminates an entire class of runtime type errors at zero performance cost — they are caught at compile time
- NeonDB serverless automatically scales database connections with demand, removing cold-start connection pool overhead

12. Testing & Development Status

12.1 Automated Test Suite

PayFlow includes TestCases.ts, a custom automated test runner covering the full lifecycle of every major feature. Tests are executed via npm test which runs npx ts-node src/TestCases.ts.

Test Category	Coverage
User Registration	Valid registration, duplicate user detection, missing field validation
Authentication	Correct credentials, wrong password, unknown userId
Balance Checks	Correct balance returned, balance after debit/credit operations
UPI Payments	Successful payment, insufficient funds, invalid UPI ID, wrong MPIN
Net Banking Payments	Successful payment, invalid IFSC code, insufficient funds
Transaction History	Correct records returned per user, empty history
Transaction Cancellation	Cancel PENDING, attempt to cancel SUCCESS, attempt to cancel FAILED

12.2 Development Status

Component	Status
Backend REST API	Complete — deployed on Vercel
Frontend SPA	Complete — deployed on Vercel
Database Schema	Complete — running on NeonDB
Console CLI (Main.ts)	Complete — runnable locally with npm run cli
Automated Tests (TestCases.ts)	Complete — 75+ test scenarios, all passing
UML Diagrams	Complete — Class, ER, Sequence, Use Case in diagrams/ folder

12.3 Future Roadmap

- WalletPayment: Add a third payment type using UPI QR codes — requires only a new factory case
- JWT Authentication: Replace session-based auth with JSON Web Tokens for stateless API security
- Rate Limiting: Implement per-user transaction rate limiting to prevent fraud patterns
- Webhook Support: Emit transaction events to external subscribers using the Observer pattern explicitly

- Admin Dashboard: A separate admin role with access to cross-user transaction analytics

13. Getting Started

13.1 Prerequisites

- Node.js v18 or above
- npm (Node Package Manager)
- A NeonDB account at https://neon.tech for the database connection string

13.2 Backend Setup

Step	Command	Notes
1. Navigate	cd payment-gateway-Backend	Enter the backend directory
2. Install	npm install	Installs TypeScript, ts-node, Express and type definitions
3. Configure	cp .env.example .env	Copy the environment template
4. Set database	Add DATABASE_URL=<neon-connection-string> to .env	Paste NeonDB connection string from your project dashboard
5. Run dev server	npm run dev	Starts Express API on http://localhost:3000
6. Run CLI mode	npm run cli	Starts the original console-based interactive application
7. Run tests	npm test	Executes all 75+ automated test scenarios

13.3 Frontend Setup

Step	Command / Action	Notes
1. Navigate	cd payment-gateway-frontend	Enter the frontend directory
2. Configure API URL	Edit js/config.js — set BASE_URL to http://localhost:3000	Points frontend at your local backend
3. Run frontend	npm run dev	Opens SPA on http://localhost:3001

13.4 Database Setup

- Create a project at https://neon.tech
- Run the provided schema.sql file in the NeonDB SQL editor to create all required tables
- Copy the connection string from the NeonDB dashboard and paste it as DATABASE_URL in your .env file

14. Conclusion

PayFlow demonstrates that the foundational principles of software engineering — OOP, SOLID, and Design Patterns — are not academic abstractions. They are practical engineering decisions that make real systems easier to build, safer to extend, and simpler to debug.

Engineering Goal	How PayFlow Achieves It
Safety	TypeScript strict mode catches type errors at compile time; encapsulation prevents credential leakage; MPIN authorization gates every payment
Extensibility	OCP + Factory Pattern mean new payment types require no changes to existing, tested code
Consistency	Singleton Pattern guarantees a single transaction store with no possibility of split state
Clarity	SRP ensures every file has one clear purpose — any developer can open any file and immediately understand its role
Testability	Interface-driven design allows any component to be replaced with a mock for isolated unit testing
Scalability	O(1) Map lookups, serverless NeonDB, and Vercel deployment make the system ready for production load

Project Summary

PayFlow is a full-stack, cloud-deployed Payment Gateway System that serves as a complete reference implementation of TypeScript, OOP, SOLID principles, and classic design patterns — all within a real, working application that any developer can clone, run, and extend.